CS 445: Project Proposal
Netids: eeroad2, jwbarry3, lucywu3

Project: Moving Least Squares
2006 Paper (MLS for image deformation):
https://people.engr.tamu.edu/schaefer/research/mls.pdf

Motivation:
This paper is a stepping-stone paper that is cited in a paper called MLS-MPM for fluid simulation, which is another SIGGRAPH paper. We would like to learn about this process to understand the mathematical architecture that leads to the MLS-MPM paper. This is a great opportunity to apply what we learned in this class to image deformation and prepare to understand modern, cutting-edge papers. We would also like to make our technique real-time, which means using shaders with either OpenGL or Vulkan.

Milestones:
1.) Read and understand the paper fully. Plan the exact architecture and set up GitHub for code sharing. (Week 1)
2.) Setting up a valid OpenGL or Vulkan context for the project with basic functionality (Week 2)
3.) Rendering a texture to the screen using shaders and low-level calls to make it very fast! (Week 3)
4.) Setting up weights based on MLS and getting basic image deformation (Week 4)
5.) Creating an animation with our system and adding a GUI system like ImGUI (Week 5)

Evaluation
We will evaluate our method based on its performance and image quality. As I understand now, MLS for image deformation is actually quite performance-intensive because a naive implementation will add the weights sequentially for every one that exists. So making this more performant might be a very interesting path. We also want smooth results, just like the paper, that can keep the underlying structure in place.

Resources
Most importantly, each person in the group needs a computer. Also, we would like to keep our implementation as accessible as possible, which means either using an older version of OpenGL to support Mac or using Multan-vk for Mac. We will need a few libraries to support this, such as GLFW (for windowing), slang (for compiling shaders), and our Vulkan or OpenGL binding libraries like volk.h. These should all be possible to get through vcpkg, which should work on Mac.

Ideas

Beyond a faithful real-time implementation of Moving Least Squares image deformation, we see a few natural extensions.

One direction is object-aware deformation: instead of wrapping the entire image, we would constrain control points (or add soft constraints) so the background stays fixed while only a selected foreground object deforms. That could come from simple user masks at first, and later from lightweight object detection or segmentation if time allows.

Another is turning the demo into a small interactive editor: drag control points live, keyframe them over time, and play back smooth deformations as animation. Our milestones already point toward ImGui and real-time shaders; an editor would make that feel like a usable tool rather than a one-off demo.

Moreover, a stretch goal is applying the same MLS machinery to animated 3D volumes or meshes. That is closer to the MLS-MPM lineage that motivated this project, but it is likely out of scope unless the 2D path goes very smoothly.

Finally, MLS-style deformation still shows up in modern creative tools and demos, which suggests the technique is worth polishing for performance and UX, not only for historical interest.

Group

eeroad2: Paper/math and MLS weight formulation; core deformation logic.
jwbarry3: OpenGL/Vulkan + GLFW setup; texture rendering pipeline.
lucywu3: Real-time shaders for MLS evaluation; ImGui editor/UI and animation/keyframing.
All members will share debugging, performance work, and the final write-up/demo. Roles may shift if we hit blockers, but each person will own substantial implementation code.

Paper notes

The paper introduces a method for object deformation that minimizes local shearing/deformation while defining a transformation f that maps a set of keypoints p to their user-transformed positions, p'.

The image is first mapped onto a regular grid. Each grid point vertex, v, is assigned a local *affine* transformation, $l_v = Mx + T$, such that (1) $w_i|l_v(p_i) - p_i'|^2$ is minimized (where p_i and p'_i are row vectors representing each control point pair). The weight factor for each constraint depends on v, so this is a v-dependent (moving) least squares problem.

We then define the global deformation function f to be $f(v) = l_v(v)$ As a point v gets closer to p_i, the weight $w_i = \frac{1}{|p_i - v|^{2\alpha}}$ approaches infinity, so f acts as an interpolant between control point maps. If $\alpha \geq 1$ then f is smooth everywhere; it is discontinuous at its control points otherwise.

Affine Transformation

The affine transformation consists of a rotation/scaling/reflection transformation **M** and a translation component **T**. Therefore, $l_v(p_i) = Mp_i + T$. Since the **T** that minimizes (1) can be computed where d**M**/d**tx** and d**M**/d**ty** are equal to 0, and this depends only on the remaining variables in (1), we can rewrite the translation component as $q^* - Mp^*$ where p^* and q^* are weighted centroids of the unwarped and warped control points. We can sub this back in, and, with $\widehat{p_i} = p_i - p^*$ and $\widehat{q_i} = q_i - q^*$ (the mean-centered control points), our minimizer becomes

(2) $\sum_i w_i |M\widehat{p_i} - \widehat{q_i}|^2$. We can use the normal equations $(A^T A)^{-1} A^T b = \bar{x}$ to derive the transformation **M** that minimizes this least squares constraint:

$M = (\sum_i \widehat{p_i} w_i (\widehat{p_i})^T)^{-1} \sum_j w_j \widehat{p_j} (\widehat{q_j})^T$. Since p is set at by the user, each time it is reset, we can just pre-compute the parts of **M** that are in terms of p and store them as scalars. More specifically, for each control point p_j, $A_j = \bar{v} B w_j \widehat{p_j}^T$ where $B = (\sum_i \widehat{p_i} w_i (\widehat{p_i})^T)^{-1}$. Then, at runtime, as q are moved, the affine transformation map f_a is $f_a(v) = \sum_j A_j \widehat{q_j} + q^*$.

Similarity Transformation

To produce per-point transformations that don't have non-uniform scaling and shearing, we need to add additional constraints to the minimizer that solves for **M**. We can start by restricting from affine to *similarity* transformations, a subset of affine transformations that only uniformly scale, translate and rotate.

To restrict **M** to be similar, we must constrain **m1** and **m2** such that **m1** dot **m2** = 0, and **m1** dot **m1** = **m2** dot **m2** (basis vectors must be orthogonal and have the same magnitude). The minimization in (2) becomes (3) $\sum_i w_i |O_i m_1 - \widehat{q_i}^T|^2$ where $O_i = \begin{bmatrix} \widehat{p_i} & -\widehat{p_i}^{\perp} \end{bmatrix}^T$. Essentially, we are solving for one column of **m** and constraining the other to be an orthogonal vector of equal magnitude. Again, we can use the normal equations to find a closed-form solution for the optimal **m1**:

$M = \frac{1}{\mu_s}\sum_i w_i \begin{bmatrix} \widehat{p_i} & -\widehat{p_i}^{\perp} \end{bmatrix}^T \begin{bmatrix} \widehat{q_i}^T & -\widehat{q_i}^{\perp^T} \end{bmatrix}$, where $\mu_s = \sum_i w_i \widehat{p_i}(\widehat{p_i})^T$, which we can again rewrite in terms of q_j:

(4) $f_s(v) = \sum_i (\frac{1}{\mu_s} A_i)\, \widehat{q_i} + q^*$

Where, like before, A_i is a scalar that is precomputed based on the defined control points:

$A_i = w_i \begin{bmatrix} \widehat{p_i} & -\widehat{p_i}^{\perp} \end{bmatrix}^T \begin{bmatrix} (v - p^*)^T & -(v - p^*)^{\perp^T} \end{bmatrix}$

Putting this all together, we can quickly find $f_s(v)$ each frame using each vertex's precomputed scalar weights.

Rigid Transformation

Further Directions

The presence of closed-form solutions for the affine, similarity and rigidity constraints makes a real-time implementation of this possible. If we want to add additional constraints, we have to ensure that there is still a sufficient analytical solution for finding local transformations.

1. **Interactive keyframe editor**
2. **Object-aware deformation**

Assuming a suitable mask is generated, we could naively deform the entire image with vanilla MLS, find the original mask's bounding curves, and deform those to re-compute the mask. However, this could waste a lot of computational time if the object is small. Instead, we could isolate the object, compute a minimal grid around it, perform MLS deformation on it, and then composite it back onto the original image, ignoring its unset background pixels when copying them back.

Finding a mask around the object isn't the main concern of this addition, and can be done simply using intelligent scissors or using object classification.

Depending on the deformation of the object, holes can form in the background image. If the image is from a video with a moving foreground subject, we can find the likely background pixels by taking the median of pixel values across all frames, and both generate an object mask (with feathering) and fill in gaps without any heavy ML.

If the foreground object is still, or the background isn't a video frame, then some in-fill technique is necessary. We could start using one of the methods described in class, and resort to diffusion or another more intensive technique if time permits.

Results (Preliminary):

Same warp of two birds (willet and loon) with Affine, Similarity and Rigid MLS constraints!

